

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ
СИСТЕМ**

Лабораторна робота №2

Реалізація схеми обміну даними між програмним забезпеченням
(SW) та апаратною частиною (HW) з використанням інтерфейсу
UART

Виконав: ст. гр. КІ-404
Стефанів В.В.
Прийняв:
Федак П.Р.

Львів 2024

Мета роботи: Розробити просту систему комунікації між програмним клієнтом та апаратним сервером, використовуючи UART для обміну повідомленнями. Перевірити коректність обміну даними, включаючи відправлення, модифікацію та зворотну передачу повідомлень. Налаштувати автоматизацію збірки, тестування та генерації артефактів за допомогою YML-файлу.

ТЕОРЕТИЧНІ ВІДОМОСТІ

UART (Universal Asynchronous Receiver-Transmitter) — це апаратний модуль або програмний інтерфейс, що використовується для передачі та прийому серійних даних між комп'ютером та іншими пристроями. UART є одним з найбільш поширених протоколів серійної передачі даних і часто використовується в мікроконтролерах, комп'ютерах та інших вбудованих системах.

Основні характеристики UART:

1. **Асинхронність:** UART не використовує спільного годинника між передавачем і приймачем, тому передача даних відбувається асинхронно, а синхронізація здійснюється за рахунок стартового та стопового бітів.
2. **Формат кадру:** Передача даних відбувається у вигляді кадрів, що складаються з:
 - Стартового біту (сигналізує про початок передачі)
 - Даних (зазвичай 8 біт, але може бути 5–9 біт)
 - Паріті біт (опціональний, для перевірки на помилки)
 - Стопового біту (сигналізує про закінчення передачі)
3. **Швидкість передачі (baud rate):** Це кількість бітів, які передаються за секунду. Обидві сторони (передавач і приймач) повинні бути налаштовані на одну й ту саму швидкість.
4. **Дуплексність:** UART підтримує як напівдуплексний (одночасна передача тільки в одному напрямку), так і повнодуплексний (передача в обох напрямках одночасно) режими.

Основні етапи роботи UART:

1. **Передача даних:** Коли дані готові до передачі, UART генерує стартовий біт, потім передає біт даних і, за потреби, париті біт, а після цього — стоповий біт.
2. **Прийом даних:** Приймач UART чекає на стартовий біт, після чого починає приймати біти даних, перевіряє їх на правильність (якщо використовується париті) та чекає на стоповий біт для завершення передачі.

ЗАВДАННЯ

1. Розробити схему обміну даними між програмним забезпеченням (SW) та апаратною частиною (HW), використовуючи інтерфейс UART.
2. Реалізувати програмний клієнт, який надсилає повідомлення до сервера. Реалізувати серверну частину, що приймає повідомлення, змінює його вміст та надсилає модифіковане повідомлення назад клієнту.
3. Створити YML-файл для автоматизації таких процесів:
 - збірка всіх бінарних файлів проєкту;
 - запуск тестів для перевірки функціональності;
 - створення артефактів, що містять бінарні файли та результати тестування.
4. Обов'язкові кроки до кожної лабораторної.

Виконання роботи

1. Код для клієнта написаний мовою веб-програмування TypeScript з використанням бібліотек React (яку використав для побудови інтерфейсу), Electron (яку використав для отримання десктоп-програми з веб інтерфейсу), а також SerialPort (яку використав для роботи з серійними портами).

Оскільки програма написана для веб-інтерфейсу, то окрім окремої програми, нею можна користуватися прямо в браузері.

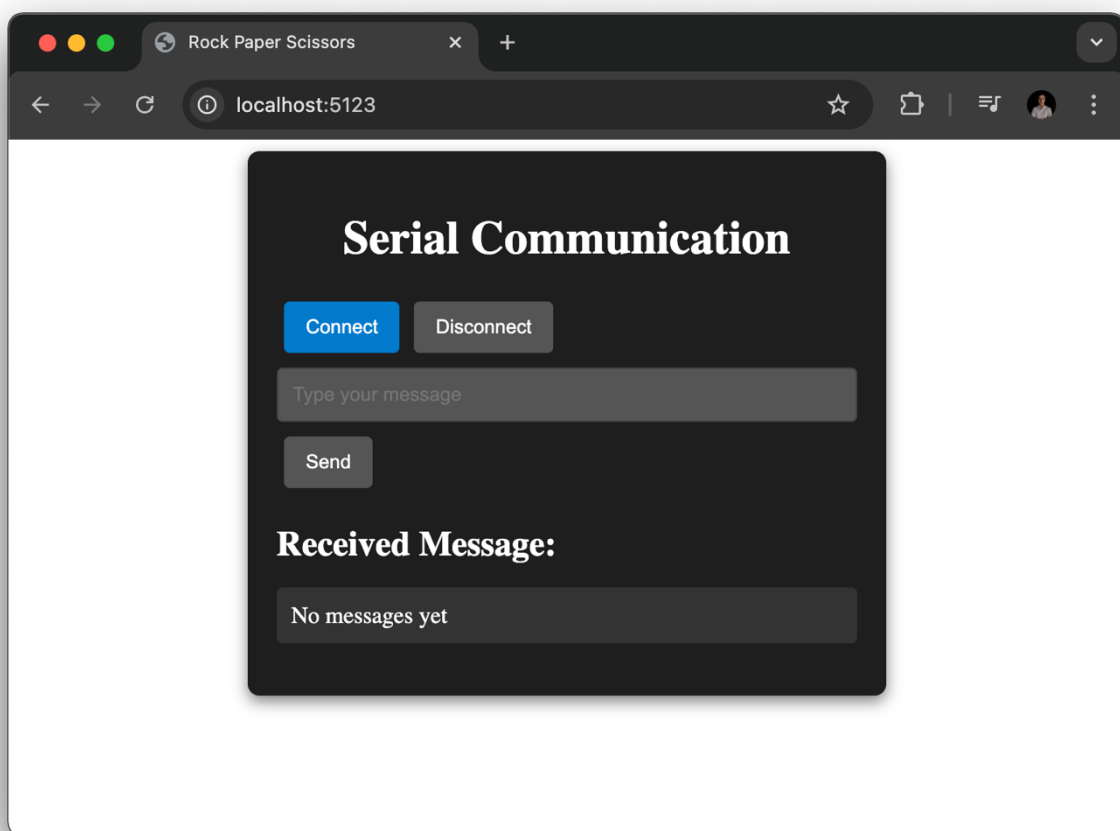


Рис.1. Клієнтська частина в браузері

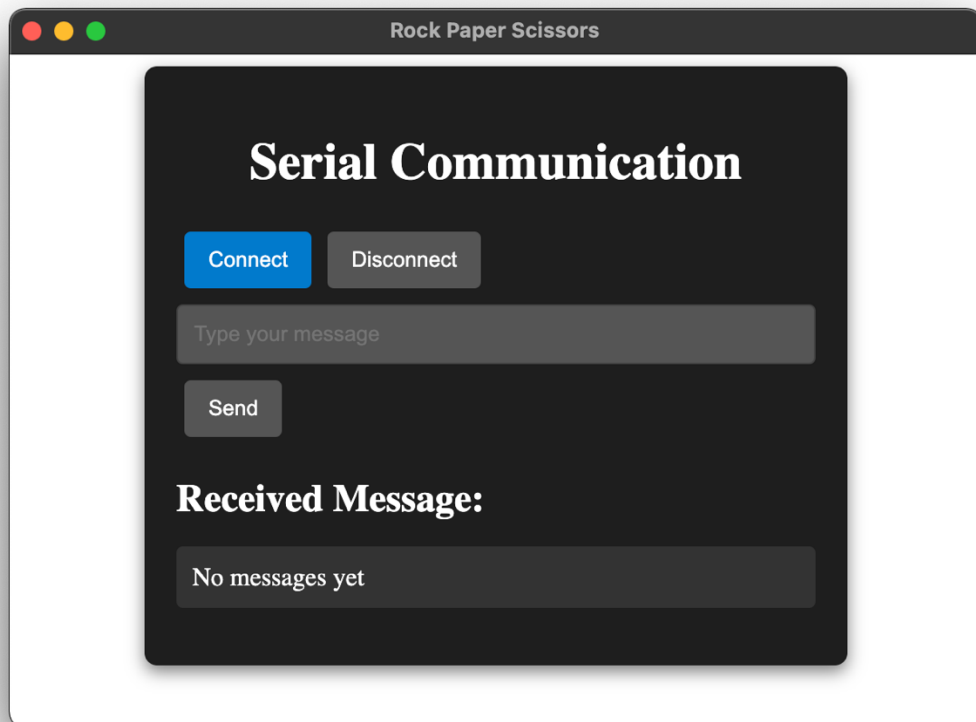


Рис.2. Клієнтська частина в десктоп-додатку

2. Написав код для серверу С-подібною мовою програмування Arduino, для якої він і написаний.

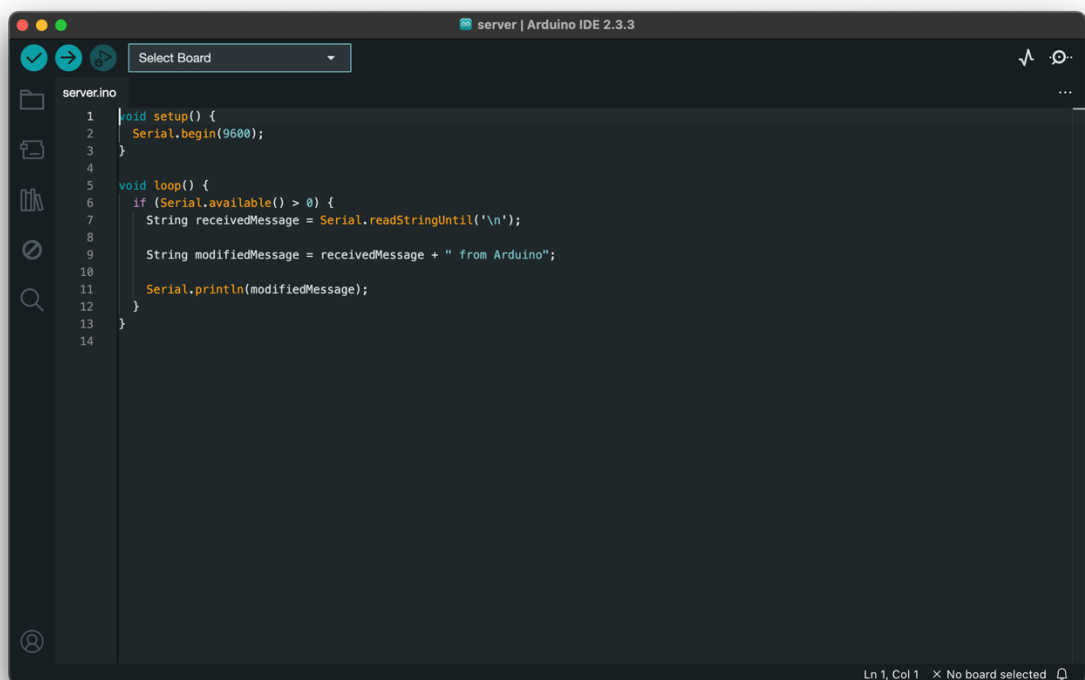


Рис.3. Код серверної частини

3. Також у ході виконання роботи були написані тести для основних частин системи, таких як React компонент, хук та контекст. Результат тестів:

```
✓ test/serial-component.test.tsx (6) 320ms
✓ test/serial-context-provider.test.tsx (1)
✓ test/use-serial-communication.test.tsx (1)

Test Files  3 passed (3)
Tests      8 passed (8)
Start at   21:02:05
Duration   5.69s (transform 397ms, setup 1.04s, collect 1.14s, tests 492ms, environment 4.56s, prepare 494ms)
```

Рис.4. Результат тестів

4. Запустимо програму та перевіримо правильність виконання коду цілої системи. Після під'єднання вводим повідомлення, яке передаватимемо, після чого натиснемо кнопку Send. Одразу після цього отримаємо відповідь сервера, тобто змінене повідомлення, до якого додано «from Arduino». Отже, робимо висновок що програма працює правильно.

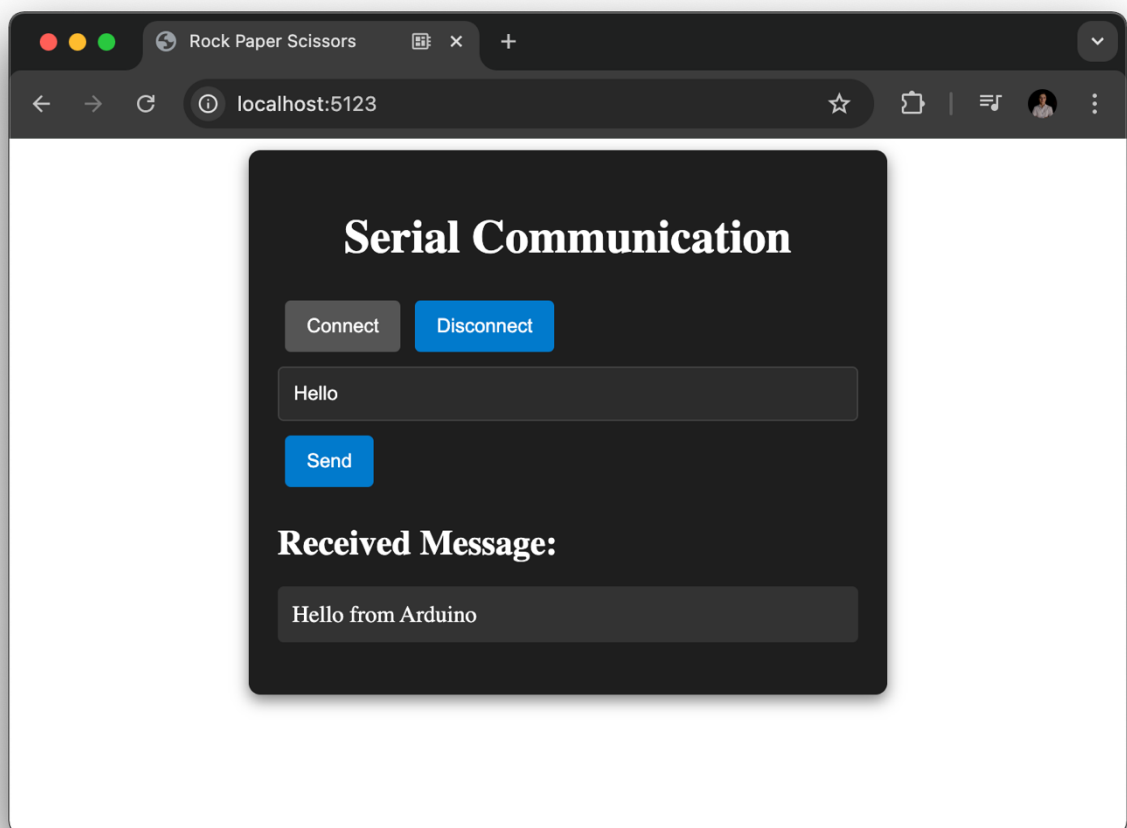


Рис.5. Виконана програма

5. Додамо конфігураційний файл для CI. У ньому вказуємо роботи які виконуватимуться. Серед них – збірка Arduino проекту, React проекту, а також тестування.

```
1  name: CI Workflow
2
3  on:
4    push:
5      branches:
6        - "*"
7    pull_request:
8      branches:
9        - "*"
10
11  jobs:
12 > build-arduino: ...
40
41 > build-react: ...
69
70 > test-react: ...
94
```

Рис.6. CI конфігураційний файл

Після цього, робимо необхідні зміни в репозиторії, далі відкриваємо Pull Request, та перевіряємо роботу CI. Оскільки всі стадії CI пройдені, робимо висновок що код написаний правильно, а також сам CI сконфігурований.

Triggered via pull request yesterday

Vitaliistf synchronize #3 `feature/develop/task2`

Status

Success

Total duration

55s

ci.yml

on: pull_request

✓ Build React Project

19s

✓ Run React Tests

21s

✓ Build Arduino Project

15s

Висновок: У ході виконання лабораторної роботи я налаштував комунікацію та обмін повідомленнями між апаратним сервером і програмним клієнтом за допомогою UART. Для розробки серверної частини використав Arduino, для клієнтської TypeScript, React, Electron. Перевінив правильність виконання з допомогою юніт-тестів, написаних з Vitest та Jest, а також експериментально. Налаштував CI для збірки, тестування та створення артефактів, а також зробив необхідні зміни до репозиторію.